

Scalable Technologies for Data Analysis

Scalable Processing: Universal Patterns and Optimization

Davide Carneiro

Mestrado em Engenharia Informática

2025/2026

Scalable Processing: Universal Patterns and Optimization

- By the end of this class you should be able to
 - Recognize and apply **universal transformation patterns** (filters, aggregations, joins, window functions)
 - Explain shuffle, parallelism, and data skew, and **anticipate** when a query will become **expensive**
 - Apply concrete **optimization strategies** in SQL/lakehouse environments
 - Decide between materialization vs. query-on-read, justifying trade-offs
 - Use evidence-based optimization through execution plans and runtime metrics

The plan

1. Intro to modern Data Platforms and scalability
2. Contract-Driven Design and Data Flow Modeling
3. Storage + table formats
4. Processing patterns + optimization 
5. Orchestration + production reliability
6. Serving: SQL/APIs/semantic layer
7. Quality + observability + cost
8. Incremental + streaming
9. ML integration
10. Governance + evolution

Previously...

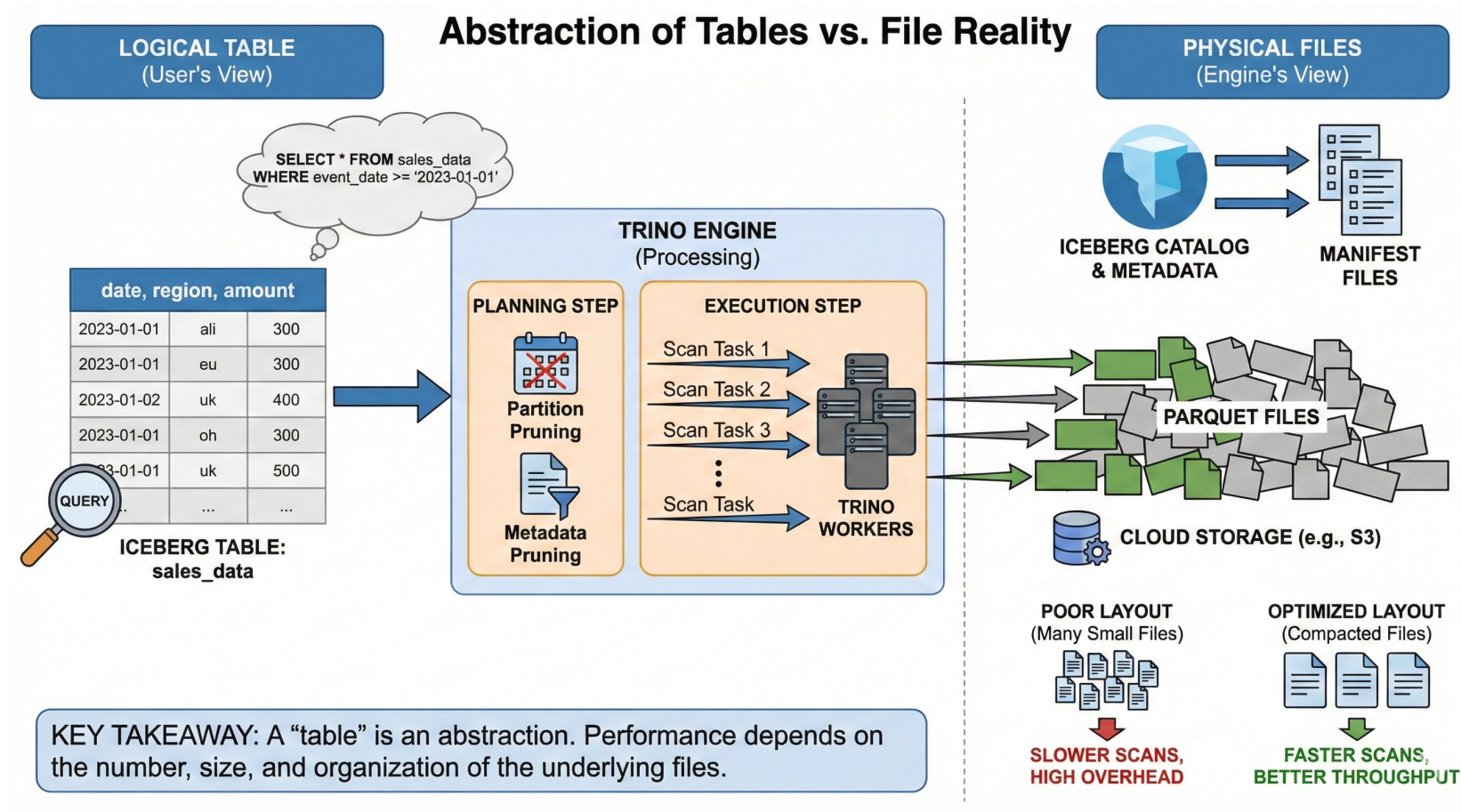
- In the previous classes, we
 - Discussed data modeling through data products, grain, keys, semantic stability, versioning...
 - Discussed data storage through Parquet, partitioning, Iceberg metadata, compaction, time travel, ...
- While modeling and storing play important roles in ensuring performance at scale, the way we structure queries also has a strong impact on their cost
 - That is, given a well-designed lakehouse, how do we transform data efficiently, and prove it?
- Today: we look at universal transformation patterns and discuss evidence-based optimization of queries

1. The basics

Warming up: Abstraction of Tables vs. File Reality

- When using Trino or other engine, we query a logical table (Iceberg table name + schema + partitions + snapshots)
 - But what Trino actually reads is a set of Parquet files referenced by Iceberg metadata (manifests / file lists)
- There are three main steps
 - Analysis
 - Planning
 - Execution
- Even with “tables”, files layout matters
 - Many small files > too many tasks, overhead, slower scans
 - Poor partitioning > more files scanned > higher cost
- Compaction improves scan efficiency by reducing file fragmentation
- Key takeaway
 - A “table” is an abstraction: performance depends on the number, size, and organization of files behind it

Warming up: Abstraction of Tables vs. File Reality



The query lifecycle

- The query lifecycle has three steps (Analysis > Planning > Execution) distributed between driver and workers
 - Driver (coordinator): understands the query, builds the plan, orchestrates stages/tasks
 - Workers (executors): perform parallel work on data splits and exchange results
- Analysis (Driver)
 - Parse SQL – build a logical representation (relations, filters, joins, aggregations)
 - Validate schemas, tables, columns, and access permissions
 - Apply basic rewrites (e.g. predicate simplification)
- Planning (Driver)
 - Optimize the logical plan into a physical plan
 - Choose join strategy (broadcast vs partitioned)
 - Decide where exchanges/shuffles happen
 - Push down filters/projections (when possible)
 - Split scans into tasks (files / splits / partitions)
 - Produce a distributed stage graph (stages + dependencies)
- Execution (Workers + Driver coordination)
 - Driver schedules tasks to workers and tracks progress/failures
 - Workers execute operators in pipelines: Scan > filter/project > join/aggregate > exchange > final output
 - Runtime adapts to reality: backpressure, spilling, skew/stragglers, retries (engine-dependent)
- When a query is slow or fails, it is useful to ask
 - Is the issue in the driver? e.g., planning takes long (too many files/partitions), metadata overhead, too complex query
 - Is the issue in the workers? e.g., heavy scanning, big shuffle, skew (one worker overloaded), spills (memory pressure)
- Most “big data” pain shows up on workers, but a fragmented lake (millions of small files) can also hurt the driver during planning

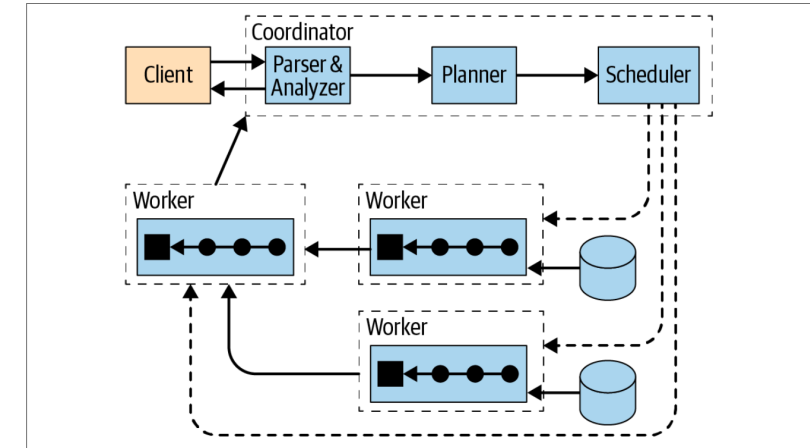


Figure 4-1. Trino architecture overview with coordinator and workers

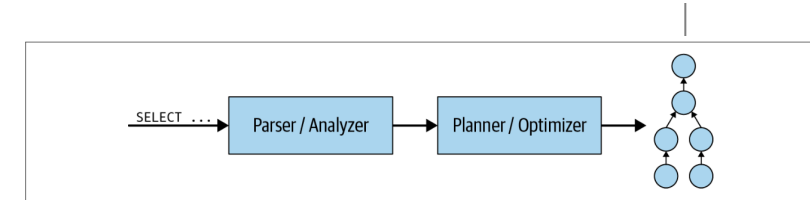


Figure 4-6. Processing a SQL query statement to create a query plan

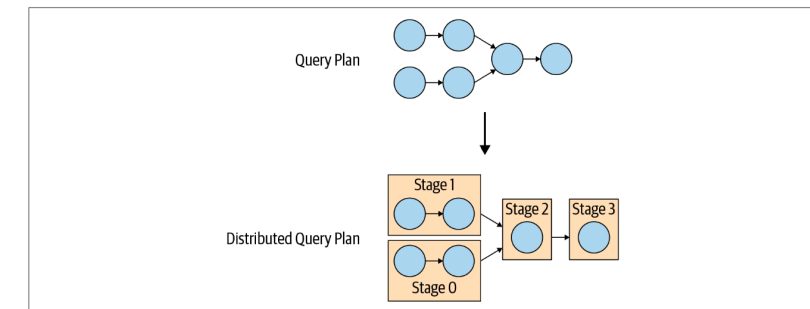


Figure 4-8. Transformation of the query plan to a distributed query plan

The query lifecycle

- In SQL, you describe **what** you want (declarative) and the engine decides **how** to compute it
- Trino is free to do
 - Predicate pushdown – move filters closer to table scans
 - Projection pruning – read only required columns from Parquet
 - Join reordering – change join order for lower cost
 - Broadcast vs. partitioned join selection – choose the cheapest distribution strategy
 - CTE inlining – WITH clauses are not automatically materialized
- You are writing a logical query, Trino executes a physical plan

CTE inlining (Common Table Expression)

- In most engines (including Trino) a WITH block is treated like a named subquery that the optimizer can inline into the main query
- Instead of running the CTE first, storing its result somewhere (**materialization**), and then reading it, Trino typically just substitutes the CTE definition into the query and optimizes the whole thing as one plan

```
WITH purchases AS (  
  SELECT event_date, customer_id, order_total_eur  
  FROM events  
  WHERE event_type = 'purchase'  
)  
SELECT event_date, sum(order_total_eur)  
FROM purchases  
GROUP BY 1;
```

Is treated as

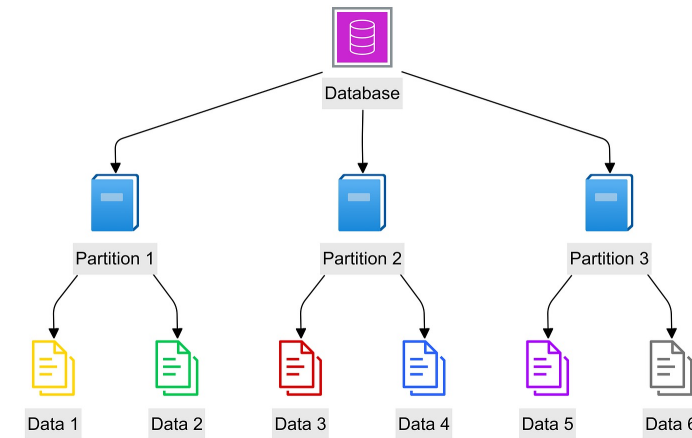
```
SELECT event_date, sum(order_total_eur)  
FROM (  
  SELECT event_date, customer_id, order_total_eur  
  FROM events  
  WHERE event_type = 'purchase'  
) purchases  
GROUP BY 1;
```

Core “mechanics” concepts

- In order to design efficient data processing queries or pipelines, we need to be aware of the underlying mechanics of a data platform
- This allows us to reason about what we are seeing in plans / metrics, and to identify points where improvements are needed and / or worth it
- In general, distributed query **cost is driven by**
 - How much data is read
 - How much data is moved (shuffle)
 - How much data must be sorted/statefully maintained
 - How balanced the work is (skew/stragglers)

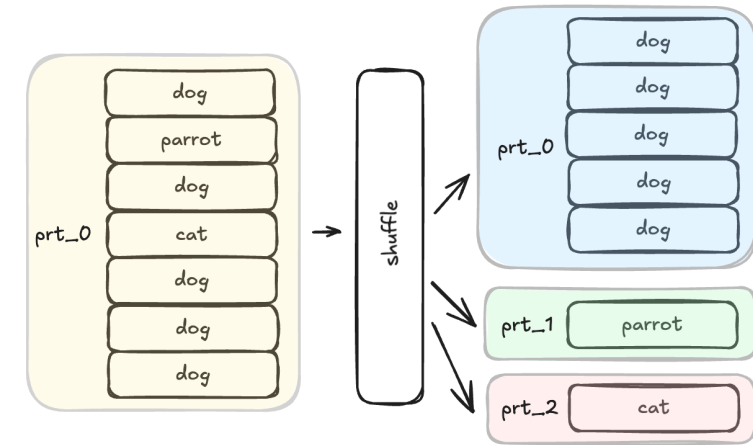
Partition (Execution Partition / Data Partition)

- A subset of the data assigned to a single parallel task (worker thread/process)
- Parallelism depends on having many partitions of roughly equal size
 - Too few partitions > low parallelism
 - Uneven partitions > **stragglers**
 - Too many partitions > more complex management, more time setting up than actually running query
- Mitigations
 - Increase partitioning granularity
 - Choose better partition keys
 - Rebalance/compact files



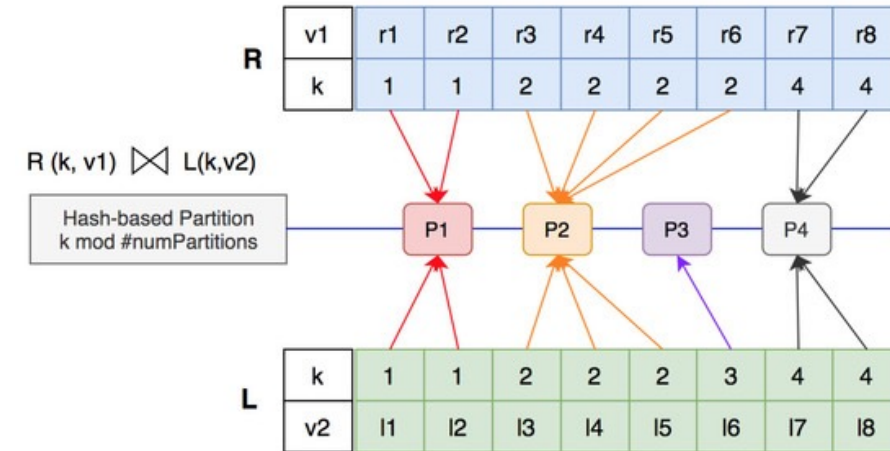
Exchange / Shuffle

- A distributed redistribution step where rows are moved across workers so that all rows with the same key end up on the same worker
 - Shuffles are often the most expensive stage (network + serialization + disk spill)
- Typical triggers
 - JOIN (on join key)
 - GROUP BY / DISTINCT (on grouping key)
 - ORDER BY (global or within partitions)
 - WINDOW functions (partition/order keys)
- Usually, it is evidenced by a big jump in “network bytes”, “spilled bytes”
- Mitigations
 - Filter/project before the shuffle
 - Pre-aggregate (reduce rows before group/join)
 - Change query shape (semi-join, EXISTS)
 - Ensure partition pruning works upstream



Skew (Key Skew / Data Skew)

- Uneven distribution of data across partitions or keys
 - Some partitions get much more data than others
- Skew kills parallelism: the slowest partition dictates overall runtime
- Common sources:
 - “Hot keys” (e.g., country='US', customer_id='unknown')
 - Null/default values
 - Time-based spikes
- Signals
 - A single stage/task runs for much longer time than others
 - Huge variance in per-task input rows/bytes
- Mitigations
 - Filter or separate hot keys (handle them in a separate branch)
 - Salt hot keys (split one key into N pseudo-keys)
 - Pre-aggregate before join/group
 - Replace join with a different semantic approach (if possible)

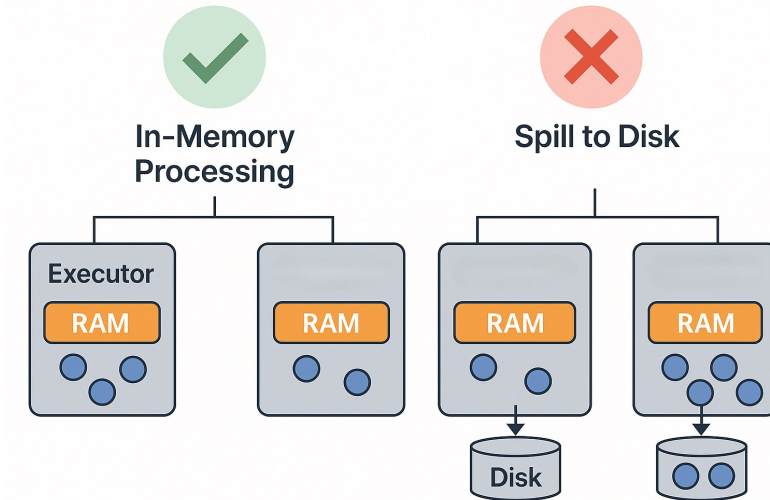


Straggler / Skew Bottleneck

- A small number of tasks take disproportionately longer than the rest, often due to skew or resource contention
- It matters as, in distributed jobs, the completion time is often the “time of the slowest task”
 - Although the idle nodes become available to accept other tasks...
- Typical signal: most tasks finish quickly, but a few linger (“tail latency”)
- Mitigations
 - Same as skew mitigations
 - Ensure adequate memory/CPU in nodes
 - Reduce intermediate size
 - Avoid global sorts

Spill (Disk Spill)

- When intermediate data does not fit in memory and is written to disk temporarily
- Spill turns a CPU/memory operation into an I/O-heavy operation, which means large slowdowns
- Signals
 - “Spilled bytes” metrics
 - Sudden runtime increase
 - Heavy temp I/O
- Mitigations
 - Reduce intermediate size (filter/project earlier)
 - Pre-aggregate
 - Increase memory (if possible)
 - Avoid wide rows
 - Avoid unnecessary columns



Details for Stage 35 (Attempt 0)

Resource Profile Id: 0

Total Time Across All Tasks: 2.1 min

Locality Level Summary: Process local: 4

Shuffle Read Size / Records: 19.3 MiB / 2009999

Shuffle Write Size / Records: 1335.5 MiB / 199980000

Spill (Memory): 0.0 B

Spill (Disk): 928.1 MiB

Associated Job Ids: 33

Cardinality

- The number of distinct values in a column or key set
- It determines the size of grouping/join state and affects partitioning distribution
- Examples:
 - `country` low cardinality
 - `user_id` high cardinality
- Signals
 - GROUP BY takes a long time
 - Distinct counts takes a long time
- Mitigations
 - Aggregate at the right grain
 - Consider approximate aggregates (where acceptable)
 - Reduce grouping dimensions or bucket by time

Row Explosion (Join Multiplication)

- When a join increases row count unexpectedly, often due to non-unique keys or many-to-many joins
- Causes massive cost increases and often wrong results
- Signals
 - Row counts after join much larger than before
 - Sums inflate
- Mitigations
 - Enforce uniqueness assumptions (contracts/tests)
 - Deduplicate dimensions
 - Join on correct keys (grain alignment)
 - Use semi-join / EXISTS when only filtering is needed

Sort / Global Order

- Ordering rows by one or more keys
- Global ordering (as opposed to partition order) requires coordination across all workers
- Sorting is expensive and global sort is often one of the worst-case operations
- Signals
 - Plans with large sort stages
 - High memory use
 - Many spills
- Mitigations
 - Avoid global ORDER BY unless necessary
 - Use LIMIT with ORDER BY carefully (Top-N optimization may help)
 - Sort within small partitions, not globally

Scan Cost (I/O Bound vs. CPU Bound)

- Whether a query is dominated by reading data (I/O) or by processing (CPU/memory)
- It matters because optimization differs
 - I/O-bound > reduce bytes read (projection, pruning)
 - CPU-bound > reduce computation (aggregation strategy, join strategy)
- Signals
 - Large “input bytes” vs. high CPU time
 - Plans show heavy compute operators
- Mitigations
 - Choose the right lever depending on which resource dominates

```
Job Counters
  Launched map tasks=1
  Other local map tasks=1
  Total time spent by all maps in occupied slots (ms)=9793
  Total time spent by all reduces in occupied slots (ms)=0
  Total time spent by all map tasks (ms)=9793
  Total vcore-milliseconds taken by all map tasks=9793
  Total megabyte-milliseconds taken by all map tasks=10028032
Map-Reduce Framework
  Map input records=1
  Map output records=1
  Input split bytes=117
  Spilled Records=0
  Failed Shuffles=0
  Merged Map outputs=0
  GC time elapsed (ms)=107
  CPU time spent (ms)=2200
  Physical memory (bytes) snapshot=144601088
  Virtual memory (bytes) snapshot=2084610048
  Total committed heap usage (bytes)=125763584
```

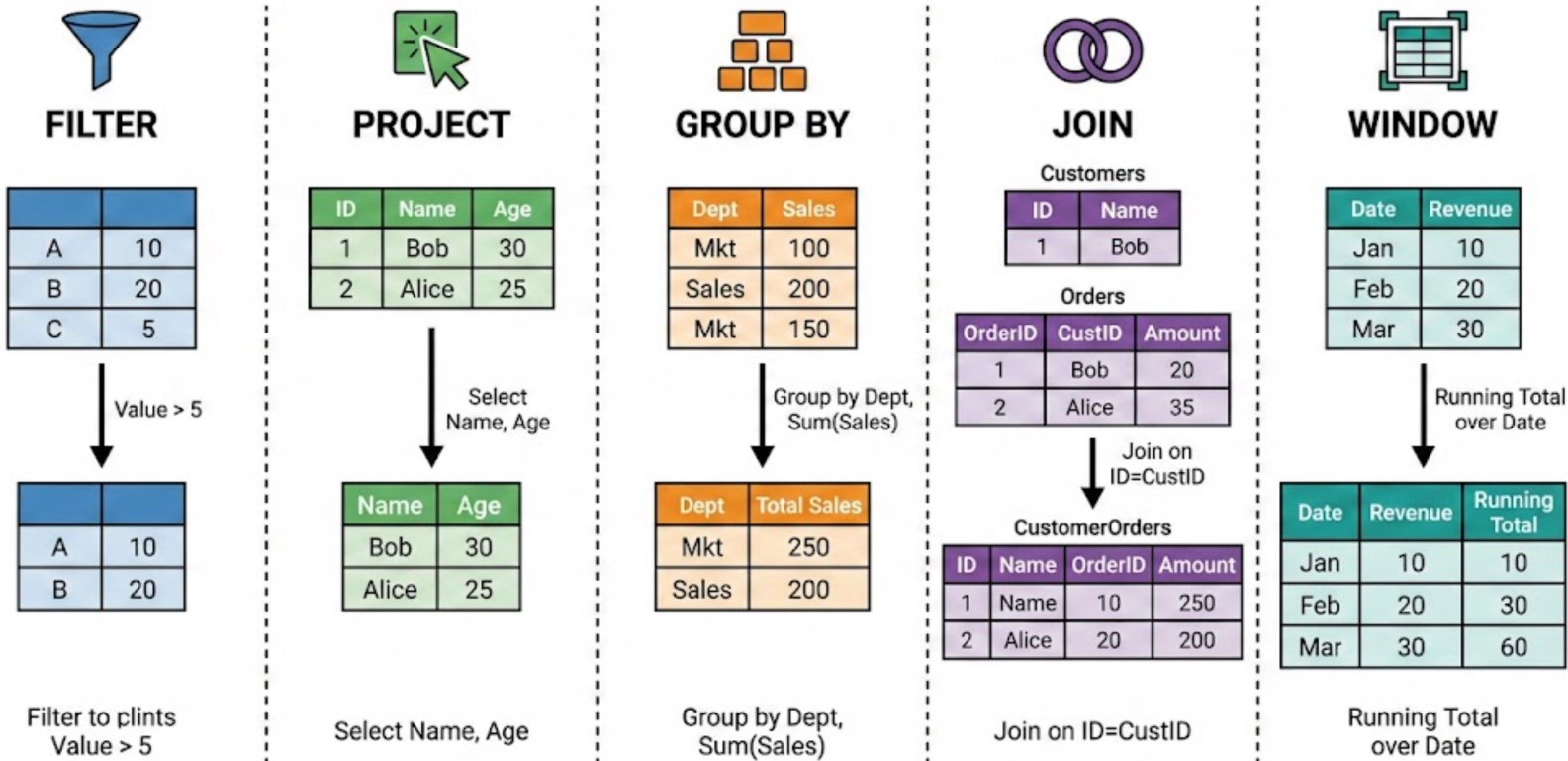
2. Data Processing

The grammar of data processing

- Almost every analytical pipeline step (batch or streaming) can be decomposed into a small set of composable operations
- These operations are “universal” because they capture how data volume changes, when distributed coordination is required, and where costs appear (I/O, network shuffle, memory pressure, and sort)
- If we learn to recognize these patterns, we can predict performance, cost, and failure modes before they happen
- Think of them as the “verbs” or functions of scalable data work:
 - **Filter** changes how many rows “survive”
 - **Projection** changes how many bytes per row you carry forward (the columns)
 - **Aggregation** changes data shape by collapsing rows into groups (often requiring redistribution)
 - **Join** combines datasets (often increasing width and sometimes exploding row count)
 - **Window** functions compute context-aware metrics over partitions, often requiring ordering (sort) and heavy coordination

The grammar of data processing

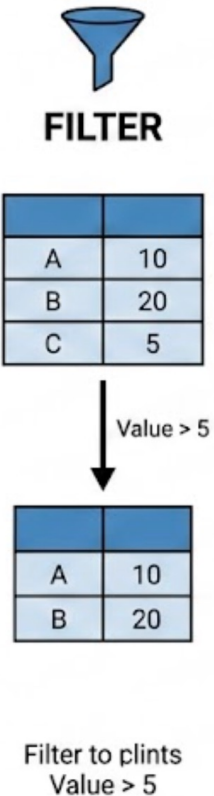
COMPARING MAIN DATA PROCESSING FUNCTIONS



(Nano Banana alert!)

Filter

- A filter removes rows that do not satisfy a predicate (e.g., `event_type = 'purchase', event_time >= now() - 7d`).
 - The optimization goal is to keep fewer rows as early as possible (especially avoid reading from file)
- Filtering is the most direct way to reduce the volume of data that every downstream operation must process
 - In distributed systems, fewer rows means less scanning, less network movement, fewer intermediate spills, and fewer failures
- It is cheap when the query engine can apply it early
 - Partition pruning (skip entire partitions by date/country/etc.)
 - Predicate pushdown (skip row groups in Parquet)
- It is **more expensive when**
 - Predicate depends on computed expressions that prevent pruning (e.g., `date(parse_datetime(ts))`)
 - Requires scanning most of the dataset anyway (low selectivity)



Projection

- Projection selects a subset of columns and/or computes derived columns (e.g., `SELECT customer_id, order_total` instead of selecting 30 columns)
 - The optimization goal is to carry fewer (unnecessary) bytes forward
- In columnar storage (Parquet/ORC), reading fewer columns can reduce I/O dramatically
 - Projection also reduces memory footprint and network payload during shuffle-heavy stages (joins/group-bys)
 - **Projection is not “cosmetic”!** In columnar systems it can be a first-order performance driver
- It is especially important when there are
 - Wide tables with many columns
 - Queries that join, sort, or aggregate (because bytes moved between workers matter)
 - “Gold” transformations where only a stable set of columns is needed



PROJECT

ID	Name	Age
1	Bob	30
2	Alice	25

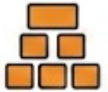
Select
Name, Age

Name	Age
Bob	30
Alice	25

Select Name, Age

Aggregation

- Aggregation reduces many rows into fewer rows by grouping keys and applying aggregates (SUM, COUNT, AVG, approx_distinct, etc.)
- Aggregations are **the main mechanism** to turn event-level data into business-level metrics (daily revenue, active users, churn rate)
 - But they can trigger expensive coordination...
- The cost usually comes from
 - If grouping keys are distributed across workers, the engine must bring all rows of the same key together (shuffle)
 - If groups are large or skewed, one worker may receive most of the data for one key (skew bottleneck)
 - Some aggregates require more state (e.g. distinct counts), which causes memory pressure
- **Partial aggregation** – many engines can aggregate locally first (e.g. MapReduce), then merge
 - This reduces shuffle volume, but only if the query structure allows it and the engine supports it
- The general notion is to pre-aggregate early when it preserves semantics, and beware high-cardinality group-bys


GROUP BY

Dept	Sales
Mkt	100
Sales	200
Mkt	150

Group by Dept
Sum(Sales)

Dept	Total Sales
Mkt	250
Sales	200

Group by Dept,
Sum(Sales)

Join

- A join merges rows from two datasets based on matching keys (e.g., events joined with customers)
- It is fundamental as almost all analytics requires enrichment, which joins encode
 - Facts gain context from dimensions (country, plan, product category)
- Joins are often the main bottleneck
 - Joins frequently require shuffle: matching keys must meet on the same worker
 - Intermediate results can become very large (wider rows + more rows)
 - A join can explode row counts if keys are not unique (many-to-many without intent)
- Joins are also relevant to data modeling as Join correctness depends on grain and key constraints
 - If dimension keys aren't unique, it can lead to duplicate facts, which leads to wrong totals and huge cost
 - If fact data isn't deduplicated, the join amplifies duplicates
- Joins should be treated as semantic operations first (grain/keys), and performance operations second (shuffle/strategy)



JOIN

Customers

ID	Name
1	Bob

Orders

OrderID	CustID	Amount
1	Bob	20
2	Alice	35

Join on
ID=CustID

CustomerOrders

ID	Name	OrderID	Amount
1	Name	10	250
2	Alice	20	200

Join on ID=CustID

Broadcast vs. Partitioned Join (Join Distribution Strategy)

- Choosing the right strategy to join can cut runtime drastically

- Broadcast join

- Replicate the right table (which should be the “small table”) to all workers, avoid shuttling the big table
 - Requires table to fit in memory in each node

```
SET SESSION join_distribution_type = 'BROADCAST';
```

- Partitioned join

- Shuffle both sides by join key and match locally
 - Each node receives matching rows from both tables
 - Potentially, “almost all” rows will have to be shuffled

```
SET SESSION join_distribution_type = 'PARTITIONED';
```

- Broadcast is good when one side is truly small

- If the “small” side isn’t really small it increases memory pressure

- Mitigations

- Keep dimensions small (select only needed columns, pre-filter)
 - Use stats (ANALYZE) and/or query hints (if available)
 - Materialize filtered dimensions for stable joins

- If you don’t know, leave it to automatic (trino will look at the row statistics and decide)



JOIN

Customers

ID	Name
1	Bob

Orders

OrderID	CustID	Amount
1	Bob	20
2	Alice	35

Join on
ID=CustID

CustomerOrders

ID	Name	OrderID	Amount
1	Bob	10	250
2	Alice	20	200

Join on ID=CustID

Window

- Window functions compute values using a “window” of related rows, defined by partitioning and ordering (e.g., rolling averages, rank, sessionization building blocks)
- It is fundamental to more advanced analytics
 - Running totals
 - Top-N per group
 - Time-based rolling metrics
 - Dedup patterns (e.g., pick latest record per key)
- However, they are expensive at scale
 - They often require sorting within partitions (ORDER BY in a window)
 - Sorting is costly (CPU + memory), and if partitions are large, it can spill to disk
- If partition keys are skewed, one partition dominates runtime
- Optimization message:
 - Use windows intentionally
 - Limit partition size (filter, pre-aggregate, or reduce scope), and avoid “global” windows unless necessary



WINDOW

Date	Revenue
Jan	10
Feb	20
Mar	30

Running Total
over Date

Date	Revenue	Running Total
Jan	10	10
Feb	20	30
Mar	30	60

Running Total
over Date

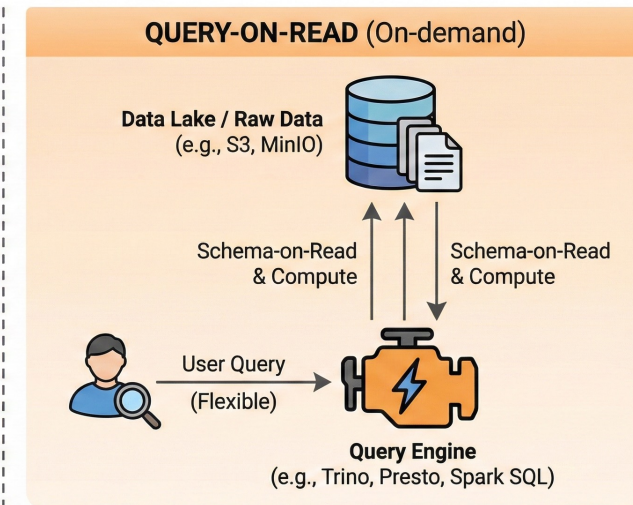
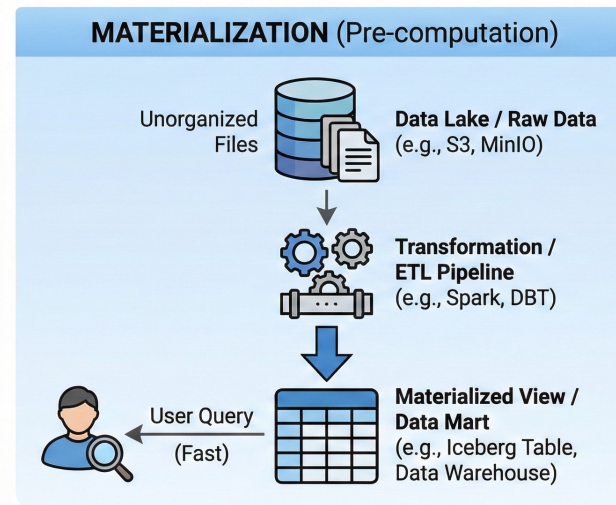
3. Optimization decisions

Materialization vs Query-on-Read

- This is fundamentally a product + platform decision
 - Where do we “pay” the compute cost (and operational complexity), and how predictable do we want consumption to be
- The core idea
 - **Materialization**: compute the result ahead of time and store it as a table (often in gold / marts), so consumers query something already prepared
 - **Query-on-read**: keep data in raw-ish/curated layers (bronze/silver), and compute the final result at query time (often via views or ad-hoc SQL)
- Both approaches can be “correct”. The “right” one depends on



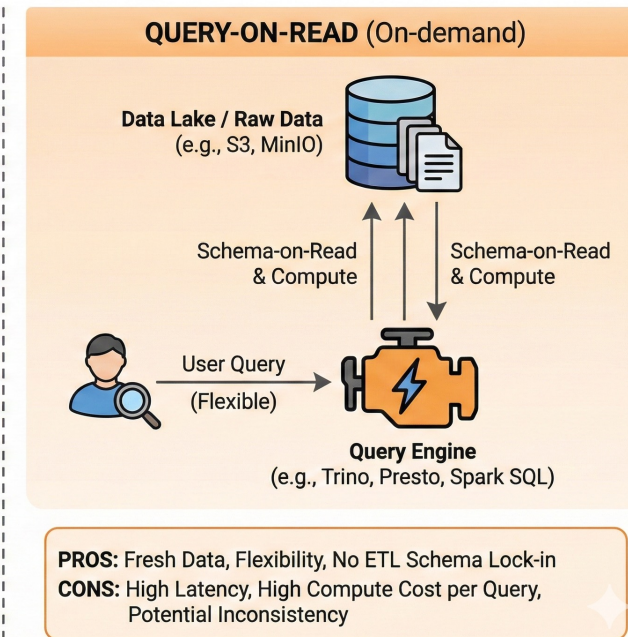
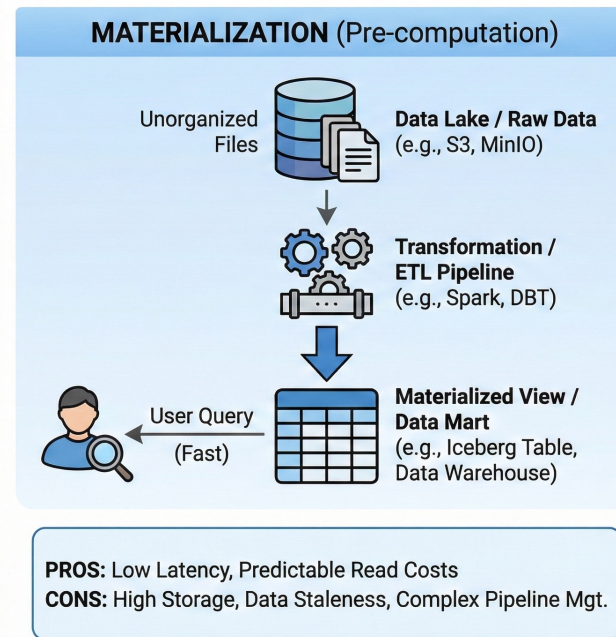
SCALABLE DATA PROCESSING: MATERIALIZATION VS. QUERY-ON-READ



Materialization vs Query-on-Read

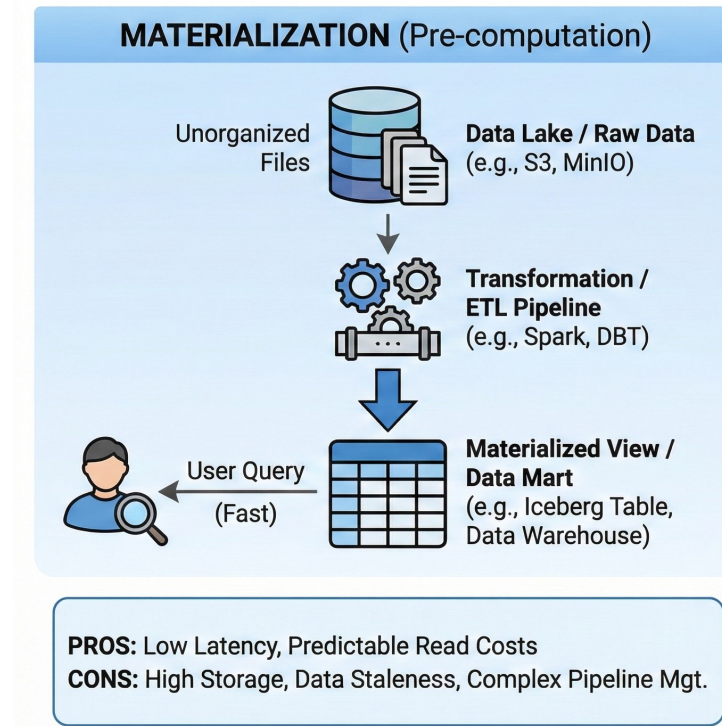
- This is fundamentally a product + platform decision
 - Where do we “pay” the compute cost (and operational complexity), and how predictable do we want consumption to be
- The core idea
 - **Materialization**: compute the result ahead of time and store it as a table (often in gold / marts), so consumers query something already prepared
 - **Query-on-read**: keep data in raw-ish/curated layers (bronze/silver), and compute the final result at query time (often via views or ad-hoc SQL)
- Both approaches can be “correct”. The “right” one depends on
 - Latency expectations
 - Concurrency (how many users/dashboards)
 - Cost profile (pay per query vs pay per refresh)
 - Semantic stability (contracts, versioning)
 - Operational maturity (scheduling/backfills/incident response)

SCALABLE DATA PROCESSING: MATERIALIZATION VS. QUERY-ON-READ



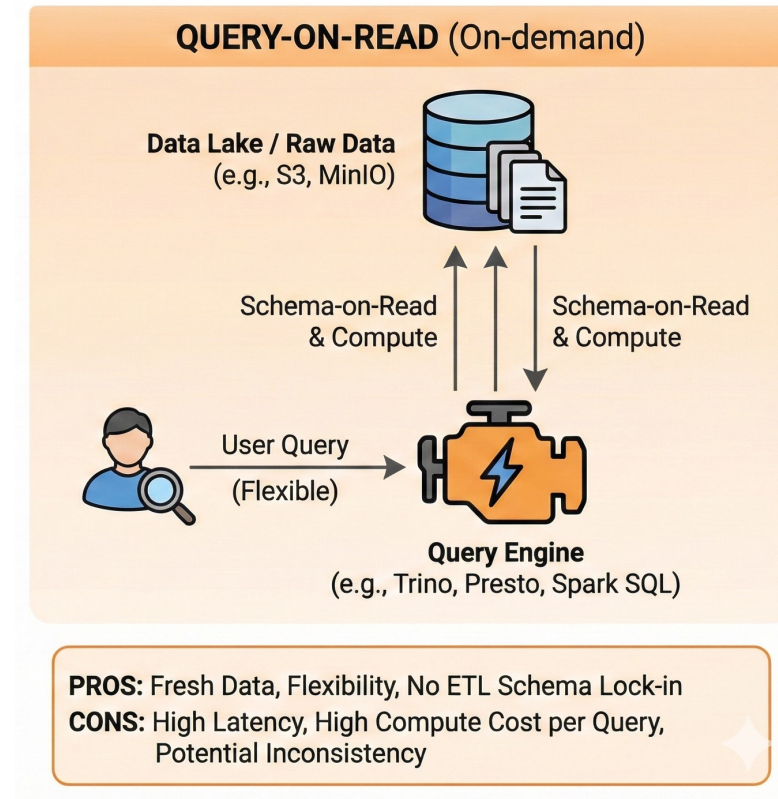
Materialization

- Materialization is not just “making a table.” It implies:
 - A refresh policy (hourly/daily/incremental)
 - Ownership and a contract (what it means, grain, keys, SLAs/SLOs)
 - Backfills and reproducibility (what happens if upstream changes or late data arrives)
 - Monitoring (freshness, row counts, anomalies, cost regressions)
- Use materialization when
 - You have many consumers querying the same logic repeatedly (dashboards, APIs)
 - You need predictable low latency
 - Definitions are stable (or versioned): “this is metric v1”
 - You want consistent governance and reuse of metrics
 - You need to cap cost by shifting compute to a scheduled job
- Typical implementation patterns
 - Gold tables (“marts”) refreshed on a schedule
 - Incremental tables (MERGE/upserts)
 - Pre-aggregations (daily rollups, top-N, session tables)
- Risks
 - Operational complexity: scheduling, dependencies, backfills
 - Storage overhead: duplicates derived data
 - Staleness: freshness becomes a product SLO you must enforce
 - Change management: schema evolution and contract compatibility become real



Query-on-read

- Use query-on-read when
 - You are exploring: metrics change frequently and you don't want to "lock in" (mostly for data science phases)
 - Latency is not strict (seconds-minutes is acceptable) or there's few data
 - Concurrency is low/moderate
 - You want maximum flexibility and minimum maintenance
 - You can tolerate variability in runtime and cost
- Typical implementation patterns
 - Views over silver tables
 - "Semantic views" (stable definitions but computed live)
 - Ad-hoc queries for analysis
- Risks
 - Cost uncertainty: repeated heavy queries get expensive
 - Performance variability: depends on cluster load, cache, file layout
 - Governance drift: metrics definitions diverge across analysts



Materialization vs Query-on-Read: decision framework

- As a rule of thumb
 - If the same expensive logic is executed 1,000 times per day by dashboards, query-on-read is often wasteful
 - If logic changes every week and only 5 people use it, materializing may create more operational burden than value
- Key questions
 - How many times per day will this be queried?
 - What's the latency expectation (interactive vs batch)?
 - Do we need a stable contract for consumers (and versioning)?
 - Is the computation expensive (shuffle-heavy joins / windows)?
 - Are we ready to operate it (refresh, retries, backfills, monitoring)?
- Deciding
 - High reuse + strict latency + stable semantics > Materialize
 - Low reuse + evolving semantics + exploration > Query-on-read
 - In between > Hybrid: materialize the expensive / more stable parts (e.g., sessionization / dedup) and keep final shaping as views

Metrics and Evidence

- Metrics and evidence answer one key question: “how do we know we’re doing well?”
- This is about building a feedback loop
 - Observe (metrics)
 - Explain (plans)
 - Change (optimization)
 - Verify (regression checks)
- Even before moving on to Flyte/observability tooling, we should internalize
 - What to measure
 - Why it matters
 - What actions it suggests

Evidence-driven optimization: the minimum set of metrics

- Query-level metrics
 - Input rows / input bytes (how much data you scanned)
 - Output rows (did you accidentally explode data?)
 - Wall time (latency)
 - CPU time (compute intensity)
 - Network/shuffle bytes (data movement cost)
 - Spilled bytes (memory pressure)
- Interpretation
 - High input bytes > improve pruning/projection/partitioning
 - High shuffle bytes > change join/aggregation strategy
 - High spilled bytes > reduce intermediate size, avoid wide joins/windows
 - Output rows unexpectedly high > modeling/keys issue (join explosion)

What to improve

- Slow but low CPU > I/O bound > filter earlier, project fewer columns, better partitioning
- High CPU > compute bound > pre-aggregate, reduce distinct/window usage, simplify expressions
- High network bytes / exchange-heavy plan > shuffle bound > reduce join sizes, join order, semi-join, pre-aggregation
- Spills > memory bound > reduce intermediate size, remove unused columns, break query into stages / materialize
- Long tail / stragglers > skew > hot key handling, salting
 - **Salting** is a technique to mitigate skew caused by “hot keys” during operations that require grouping or co-locating rows by key by artificially spreading one overloaded key across multiple partitions so that the workload becomes parallel again

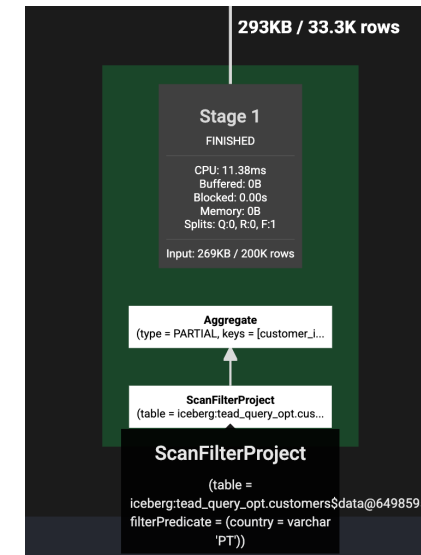
How to observe

- While there are some nice tools to use, for starters we can look at some standard elements
- Execution Plans
 - EXPLAIN tells what operators exist (joins, aggregates, exchanges)
 - EXPLAIN ANALYZE adds runtime metrics per stage/operator (engine-dependent)
- Engine UI / Query history (Trino Live Plan UI)
 - Processed rows/bytes
 - Stage times
 - Peak memory / spills
 - Distribution strategy (broadcast vs partitioned)
- Storage layer signals
 - Number/size of files read (small files penalty)
 - Partition pruning effectiveness
 - Compaction history (Iceberg metadata)
- Pipeline-level (we'll see this later with Flyte)
 - Success/fail rates, retries
 - Duration trends
 - Cost per run
- Freshness SLO

Trino Live Plan operators

- TableScan / ScanFilter / ScanFilterProject
 - Reads data from the connector (Iceberg > Parquet on MinIO), optionally applying filters and selecting only needed columns
 - This is where **predicate pushdown** and column pruning tends to happen

```
WITH clients AS (  
  SELECT * FROM customers  
)  
SELECT DISTINCT(customer_id)  
FROM clients  
WHERE country = 'PT'
```



Trino Live Plan operators

- Filter
 - Applies a predicate to rows after they have been produced by a child node
 - If there is a Filter above a TableScan instead of inside ScanFilterProject, it may indicate the filter wasn't pushed down (missed pruning opportunity)
- Project
 - Produces/renames/derives columns (e.g., computes expressions, selects a subset)
 - Projection is cheap, but it can affect downstream cost by changing row width
 - If projection isn't pushed down, you might still read extra Parquet columns

Trino Live Plan operators

- Aggregation (Partial / Intermediate / Final Aggregate)
 - Trino often executes aggregations in phases
 - Partial Aggregate: local aggregation on each worker (reduces data early)
 - Intermediate Aggregate: merges partial results (usually after an exchange)
 - Final Aggregate: produces final grouped result
- Join nodes (InnerJoin / LeftJoin / RightJoin / FullJoin)
 - Combines two streams based on join criteria. Common join strategy hints
 - REPLICATED (broadcast join): small/right side is copied to all workers
 - PARTITIONED: both sides shuffled by join key
 - Joins are a primary source of shuffle, memory usage, and row explosion. Watch for Join output rows unexpectedly larger than inputs (semantic bug / duplicate keys)

Trino Live Plan operators

- LocalExchange
 - Re-partitions data within a stage (within the worker pipeline set), typically to prepare for an operator that needs specific distribution (like grouping or joining).
Common partitioning modes
 - HASH: partition by hash(key) > used for group-by/join distribution
 - SINGLE: gather to one stream (often for final operations or small builds)
 - ROUND_ROBIN: distribute evenly without a key (less common)
 - It signals data movement / repartitioning
- RemoteExchange
 - Data is moved across the network between stages (this is the real “shuffle”). It is where we pay network + serialization costs. Watch for large exchange bytes, long blocked time, skew/stragglers

Trino Live Plan operators

- Output: Final result being returned to the client
 - Usually not the bottleneck unless result set is huge.
- Sort: Orders rows (global or within partitions)
 - Sorting is expensive and can spill to disk.
 - Watch for sort on large datasets without LIMIT
- TopN / TopNRanking
 - Efficient “ORDER BY ... LIMIT N” or “Top-N per group” patterns
 - Often cheaper than full sort.
- Limit: Stops after producing N rows
 - LIMIT alone doesn't always reduce scan cost unless combined with ordering or the engine can short-circuit early
- DistinctLimit / MarkDistinct: Operators used to implement DISTINCT efficiently
 - DISTINCT can be shuffle-heavy and memory-heavy
- Window: Executes window functions (row_number, rank, rolling windows)
 - Often implies partitioning + sorting. Can be expensive
 - Watch for large partitions (skew) and spill.
- Union / Exchange for UNION ALL : Combines streams
 - Usually straightforward, can increase volume
- Values: A small in-memory constant table (literal rows). Often appears for small helper constructs or optimizations

What “healthy” looks like

- Healthy generally means meeting agreed SLOs
 - Problems arise when there are SLO violations or negative trends
- Freshness
 - Last successful update time
 - Data arrival delay vs expected
- Completeness
 - Expected row counts vs actual (by partition/day)
 - Missing partitions
- Correctness signals
 - Schema contract compliance
 - Key uniqueness (where applicable)
 - Null rates for critical fields
- Performance and cost
 - Runtime duration trend
 - Scanned bytes trend
 - Spilled bytes occurrences
 - Cost per run / cost per question



Scalable Technologies for Data Analysis

Scalable Processing: Universal Patterns and Optimization

Davide Carneiro

Mestrado em Engenharia Informática

2025/2026